You said:

My ericwtools are already the version you asked for. But i am on linux so i have the linux version.

ChatGPT said:

Perfect! Linux is absolutely fine — **the feature is not Windows-specific**. I should have included Linux instructions from the start. 😄

The QSS generator should work with the Linux build of `ericw-tools 2.0.0-alpha11`. It looks for the executable named `light` rather than `light.exe`.

Linux setup

1. Download the Linux x86-64 build of `ericw-tools 2.0.0-alpha11` from the official releases ↗.
2. Extract it into a separate directory. Keep your existing tools installation untouched.
3. Make sure the `light` executable has permission to run:

   ```
   chmod +x /path/to/ericw-tools/light
   ```

4. In Blender, open **N → Quake → 01 · Project & Source → External Tools**.
5. Set the ericw-tools directory to the folder containing `light`.
6. Check that QSS detects the executable.

Then run the test

- Open **Generate Decoupled Lightmaps…**
- Select a small test BSP.
- Set **World Units per Luxel** to `16.0` for the first run. We can try `1.0` afterward.
- Generate the new BSP.

The original BSP remains untouched. QSS writes a separate `_decoupled.bsp` file.

One thing I want to verify: does your Linux `light` executable actually report support for `-world_units_per_luxel`?

You can check in a terminal:

```
/path/to/ericw-tools/light -help
```

Look for `-world_units_per_luxel`. If it's listed, we're ready. If not, send me the output and we'll identify the mismatch before running the test.

You said:

its listed

ChatGPT said:

Excellent! 😄 That confirms your Linux `light` executable recognizes the option we need.

We're ready for the first test!

Let's generate your first decoupled BSP

1. Open Blender and go to **N → Quake → 01 · Project & Source → External Tools**.
2. Confirm QSS detects your Linux `light` executable.
3. Click **Generate Decoupled Lightmaps...**
4. Select a small test BSP.
5. Set **World Units per Luxel** to `16.0` for the first run.
6. Start generation.

What to check afterward

Open Blender's Text Editor and select:

```
QSS_Decoupled_Lightmap_Generation
```

We want to see:

- `VERIFICATION: PASS`
- A non-zero `DECOUPLED_LM bytes` value
- A non-zero number of decoupled face records

Then import the generated `_decoupled.bsp` into QSS.

Send me the generation report, even if everything passes. Then we'll test the actual import and inspect how QSS handles the decoupled lightmap UVs. That's where things get exciting! 🚀

You said:

VERIFICATION: PASS
DECOUPLED_LM bytes: 1600
Decoupled face records: 40
Other BSPX lumps: DECOUPLED_LM
Output .lit sidecar: present

ChatGPT said:

Generation successful 🚀

That's exactly the result we wanted!

Your report confirms:

Check	Result
BSP verification	PASS
`DECOUPLED_LM` size	1,600 bytes
Decoupled face records	40
Record size	40 bytes each
Other BSPX lumps	None
`.lit` sidecar	Present

The numbers add up perfectly: 40 records × 40 bytes = 1,600 bytes.

Your Linux `ericw-tools` build has successfully generated decoupled lightmap data for 40 faces. This is our first real compiler-generated result, beyond synthetic test fixtures.

Next: import it into QSS

1. Import the generated `_decoupled.bsp` into Blender.
2. Check whether the import completes without errors.

3. Inspect the imported mesh and lightmap UVs.
4. If possible, select one of the 40 decoupled faces and check its UV layout.

Please don't export or overwrite anything yet. First, I want to establish that QSS reads the compiler-generated data correctly.

Tell me whether the import succeeds, and whether you see any warnings in Blender's console.

You said:

imported fine. Mesh and lightmap uvs look fine. But i dont see a difference to undecoupled?

ChatGPT said:

That's actually a good sign! It means the import hasn't obviously broken the mesh or its UVs. But you're right: **we need to establish whether the decoupled lightmap projection is genuinely different, not just whether it imports successfully.**

There are two reasons why you might not see a visual difference:

1. **The geometry may not need much decoupling.** On simple walls and floors, the lightmap projection can look almost identical to the conventional projection.
2. **The current UV display may hide the difference.** Decoupled lightmap UVs are generated from independent world-space projection axes. They don't necessarily look dramatically different on every face.

Let's run a more meaningful comparison

I'd like you to try one quick check:

1. Keep both BSPs available: the original and the generated `_decoupled.bsp`.
2. Import both into Blender, ideally into separate collections.
3. Compare the lightmap UVs of a face that is angled, rotated, or otherwise awkwardly oriented.
4. Look for differences in UV orientation, scale, or island placement.

One important question: when you say the lightmap UVs look fine, are you looking at the actual lightmap UV layer in Blender's UV Editor, or at the rendered lighting on the mesh?

Those are two different tests. The UV Editor tells us whether QSS is interpreting the decoupled projection; rendered lighting tells us whether the samples are being applied correctly.

You said:

shouldnt i just use the lighmap painting feature to paint a higher resolution lightmap onto one of the faces?

ChatGPT said:

Yes! That's actually the better next test. 😄

The whole point of decoupled lightmaps is to let a face have its own lightmap resolution and projection, independently of its texture mapping.

Let's use QSS's lightmap painting feature to test that.

Test: paint one decoupled face

1. Select the imported decoupled BSP.
2. Enter the QSS lightmap painting workflow.
3. Choose one of the 40 decoupled faces.
4. Paint a small, obvious pattern on it — something asymmetric, like a diagonal stripe or a cross.
5. Inspect the result in Blender.

What we're testing

- Does QSS identify the face's decoupled lightmap dimensions?
- Does painting address the correct lightmap samples?
- Does the painted pattern stay on the intended face?
- Does the higher-resolution lightmap retain its detail?

One important distinction: `16 WU/luxel` is the compiler's initial lightmap density. Painting alone won't necessarily increase that resolution. We need to check whether QSS supports editing or resizing the decoupled face's sample grid.

Start with one face and don't export over your test BSP. If you tell me what resolution or brush settings the painting panel offers, I can guide the next step precisely.

You said:

should i import the decoupled bsp with a different working lightmap resolution?

ChatGPT said:

For this first test, **no — keep the working lightmap resolution at its default/current setting.**

We want to isolate whether QSS correctly handles the `DECOUPLED_LM` data generated by ericw-tools.

There are two separate things:

- **Compiler lightmap resolution:** The density used to generate the decoupled samples (16 WU/luxel in your test).
- **QSS working lightmap resolution:** The resolution used by QSS for its editable lightmap representation.

Changing both at once would make it harder to identify problems.

My recommended test sequence

1. Import the decoupled BSP at the default working resolution.
2. Paint one decoupled face.
3. Export to a new BSP.
4. Re-import that BSP and check whether the painted detail survives.

After that, we can test importing at a different working resolution and determine whether QSS preserves or resamples the original decoupled data correctly.

The key requirement: changing the working resolution must not silently destroy the original decoupled projection or sample layout.

You said:

should i export as classic or enhanced?

ChatGPT said:

Choose Enhanced BSPX export.

For this test, we want to preserve the decoupled lightmap data rather than fall back to conventional lightmaps.

- **Enhanced:** Preserves `DECOUPLED_LM` and its associated lightmap samples.
- **Classic:** Creates conventional fallback lightmaps for compatibility, but cannot preserve the full decoupled functionality in a vanilla BSP29 workflow.

Test procedure

1. Export as **Enhanced** to a new filename.
2. Re-import the exported BSP.
3. Check the lightmap UVs.
4. Check whether your painted detail survived.
5. Verify that `DECOUPLED_LM` is still present.

Keep the original generated BSP as your reference.

Let's see whether the complete decoupled import → paint → export → re-import workflow works!

You said:

i reimported and the lighmap is spot on. Looks like 16mu per pixel to me.

ChatGPT said:

YES! 😁 That's a very encouraging result.

If it looks like **16 world units per luxel**, then the generated lightmap density appears to have survived the complete workflow:

Import → Enhanced Export → Re-import passed visually

We've now demonstrated several important things:

- QSS successfully reads compiler-generated `DECOUPLED_LM`.
- The imported lightmap UVs look correct.
- Enhanced export preserves the decoupled lightmap representation.
- Re-import produces the expected lighting appearance.

One more important test

I'd like to verify that the lightmap is genuinely editable, not merely preserved.

Can you paint a visible change onto one of those decoupled faces, export as Enhanced, and re-import once more?

If that survives, we'll have tested the critical editing round-trip rather than just data preservation.

We're getting somewhere exciting here! 🚀

You said:

i already did this. The changes are reimported perfectly.

ChatGPT said:

That's fantastic! 🚀 Then we've passed a much more significant test than simply importing a decoupled BSP.

Decoupled lightmap editing round-trip successful

What we've verified on a real BSP

Test	Result
ericw-tools generation	PASS
`DECOUPLED_LM` detection	PASS
QSS import	PASS
Lightmap UV interpretation	PASS
Enhanced BSPX export	PASS
Re-import	PASS
Painted changes preserved	PASS

This is our first successful real-map validation of the decoupled lightmap editing pipeline.

What I'd like to test next

One remaining important question is whether QSS correctly handles *different decoupled resolutions on different faces*.

We should generate another test BSP using:

- `16 WU/luxel` — our current baseline.
- `8 WU/luxel` — twice the linear resolution.
- Optionally `4 WU/luxel` — four times the linear resolution.

Then import, paint, export Enhanced, and re-import again.

This would help establish that QSS handles varying sample dimensions and densities, not just one consistent compiler setting.

I would keep the current successful test BSP as our reference and never overwrite it.

And importantly: I won't mark the feature fully validated until we also test the classic fallback and an actual engine load.

You said:

where can i choes the wu/luxel setting?

ChatGPT said:

You set it in the **Generate Decoupled Lightmaps** operator, not in the import or export settings.

1. Open **N → Quake → 01 · Project & Source → External Tools**.
2. Click **Generate Decoupled Lightmaps…**
3. In the file-selection dialog, look for **World Units per Luxel**.
4. Change it from `16.0` to `8.0`.
5. Confirm the BSP file and run the operation.

For our next test, use **8.0 WU/luxel**.

That gives twice the linear lightmap resolution of 16 WU/luxel, or approximately four times as many samples per face area.

Keep the original BSP and your successful 16 WU/luxel result untouched.

You said:

8.0 works too!

ChatGPT said:

Brilliant! 😁 That is another important milestone.

We've now confirmed that **both 16 and 8 WU/luxel work** with the real-map generation and QSS workflow.

At 8 WU/luxel, the lightmap has:

- 2× the linear resolution compared with 16.
- 4× the samples per unit of face area.
- More room for fine lighting detail.

Next test: 4.0 WU/luxel

That will give us another useful resolution step:

Setting	Relative samples per face
16 WU/luxel	1×
8 WU/luxel	4×
4 WU/luxel	16×

Before we move on: **did you also paint, Enhanced-export, and re-import the 8 WU/luxel BSP, or have you only verified generation and import so far?**

I want to distinguish compiler success from a complete editing round-trip.

You said:

also painted and reimported. 4 works too..

ChatGPT said:

Now we're cooking! 😁🚀

That's a really strong result. We have successfully tested three different decoupled lightmap densities on your real BSP.

Real-map editing round-trip verified at three resolutions

WU/luxel	Relative sample density	Result
16	1×	PASS
8	4×	PASS
4	16×	PASS

At all three resolutions:

- Compiler generation works.
- QSS imports the decoupled lightmaps.
- Painting works.
- Enhanced export preserves the changes.
- Re-import reproduces the result.

What this tells us

We've now demonstrated that QSS can handle real compiler-generated decoupled lightmaps at multiple resolutions, including editing and persistence through Enhanced BSPX export.

That's a major validation milestone for v0.16.1.

Two remaining tests before I call this feature validated

1. **Classic export:** Check that the conventional fallback remains usable in Ironwail.
2. **Resolution independence:** Check whether changing QSS's working lightmap resolution affects the original decoupled sample dimensions or projection.

I'd tackle Classic export next, using a copy of your successful 4 WU/luxel BSP. Keep the Enhanced version untouched.

This is excellent progress!

You said:

ok what to do?

ChatGPT said:

Let's test **Classic BSP export and Ironwail compatibility** now. We'll use your successful 4 WU/luxel BSP.

Test 1 — Export Classic

1. Open Blender with your successfully edited 4 WU/luxel BSP.
2. Save a backup of your `.blend` file.
3. Open QSS export settings.
4. Select **Classic BSP29** (not Enhanced BSPX).
5. Export under a new filename, for example:
   `test_decoupled_classic.bsp`
6. Do not overwrite your Enhanced BSP.

Test 2 — Check the exported BSP

We want to establish that:

- The exported BSP is valid.
- Decoupled faces have conventional fallback lightmaps.
- The exported lightmap samples are within valid ranges.

Test 3 — Open in Ironwail



1. Put the exported BSP into your test map directory.
2. Launch Ironwail.
3. Load the map using its console:

   ```
   map test_decoupled_classic
   ```

4. Inspect the areas you painted, especially the face with the strongest visible changes.

Report back

I need three things:

- Does the BSP load in Ironwail?
- Does the lighting look correct?
- Are your painted changes visible?

Important: Classic export tests the vanilla-compatible fallback. It does not test whether Ironwail understands `DECOUPLED_LM` itself. We'll keep those two things separate.

Let's see how it performs!

You said:

bsp loads in ironwail
lighing looks correct
my painted changes are visible but less detailed.

ChatGPT said:

Excellent! That's exactly the result we wanted. 🚀

Classic BSP fallback verified in Ironwail

Your results make technical sense:

- **BSP loads:** Classic BSP29 export is valid and compatible with Ironwail.
- **Lighting looks correct:** The conventional fallback lightmaps are being generated correctly.
- **Painted changes are visible:** QSS successfully transfers your edits into the classic lightmap representation.
- **Less detail:** Expected. The classic fallback uses conventional lightmap resolution and atlas constraints, so it cannot necessarily preserve the full resolution of the decoupled representation.

What we've now demonstrated

Capability	Result
ericw-tools DEC generation	PASS
QSS DEC import	PASS
Painting at 16, 8 and 4 WU/luxel	PASS
Enhanced export and re-import	PASS
Classic BSP29 export	PASS
Ironwail compatibility	PASS
Painted edits in classic fallback	PASS

One final technical test

I want to verify that changing QSS's working lightmap resolution does not accidentally destroy the original decoupled sample layout.

We'll test that separately, without risking your successful BSPs.

My recommendation: preserve all three test files (16, 8 and 4 WU/luxel), plus the Classic export. These are now valuable real-world regression fixtures.

This is a substantial milestone for QSS v0.16.1.

You said:

Cheers! :D But lets get back to work. :D

ChatGPT said:

Cheers! 😁 And well deserved — we've made real progress today.

The decoupled lightmap pipeline has passed some very meaningful real-world tests. Now let's turn those findings into solid code.

My suggestion is to tackle the **working-resolution independence test** next, then review the implementation for anything that needs tightening before we move toward v0.16.2.

One question: do you want to continue hardening the decoupled lightmap system, or move on to the next feature on the QSS roadmap?

You said:

what do you mean by hardening?

ChatGPT said:

By **hardening**, I mean making the feature robust enough that we can trust it on different Quake maps, not just the test map we've successfully used.

In practical terms, I'd focus on five things:

1. Resolution independence

Make sure changing QSS's working lightmap resolution doesn't accidentally alter the original decoupled sample dimensions, projections or offsets.

2. Painting reliability

Test painting across:

- Different face sizes.
- Different orientations.
- Different decoupled resolutions.
- Faces sharing conventional lightmap atlas regions.

3. Export safety

Verify that Enhanced and Classic exports:

- Preserve painted samples.
- Handle all supported lightmap styles.
- Never produce invalid sample offsets.
- Don't silently discard unsupported data.

4. Import validation

Improve error reporting for malformed or incomplete `DECOUPLED_LM` data, rather than allowing partially valid data to cause problems later.

5. Regression tests

Turn what we just tested manually into automated tests wherever possible, so future QSS changes don't break this functionality.

My recommendation

I'd start with **resolution independence**.

It's particularly important because QSS has its own editable lightmap representation, while decoupled lightmaps carry independent dimensions and projection axes.

We need to make sure those two systems cooperate without one accidentally overwriting the other.

Then we can move on to the next major feature.

In short: hardening means making the new functionality dependable, not just making it work once. 😁

You said:

the decoupling takes pretty long. What exactly is happening here? Shouldnt it just be an upscaling of the lightmap atlas?

ChatGPT said:

Good question! And there's an important distinction here: **decoupled lightmaps are not simply an upscaled lightmap atlas.**

What ericw-tools is doing is considerably more involved.

1. What ordinary Quake lightmaps do

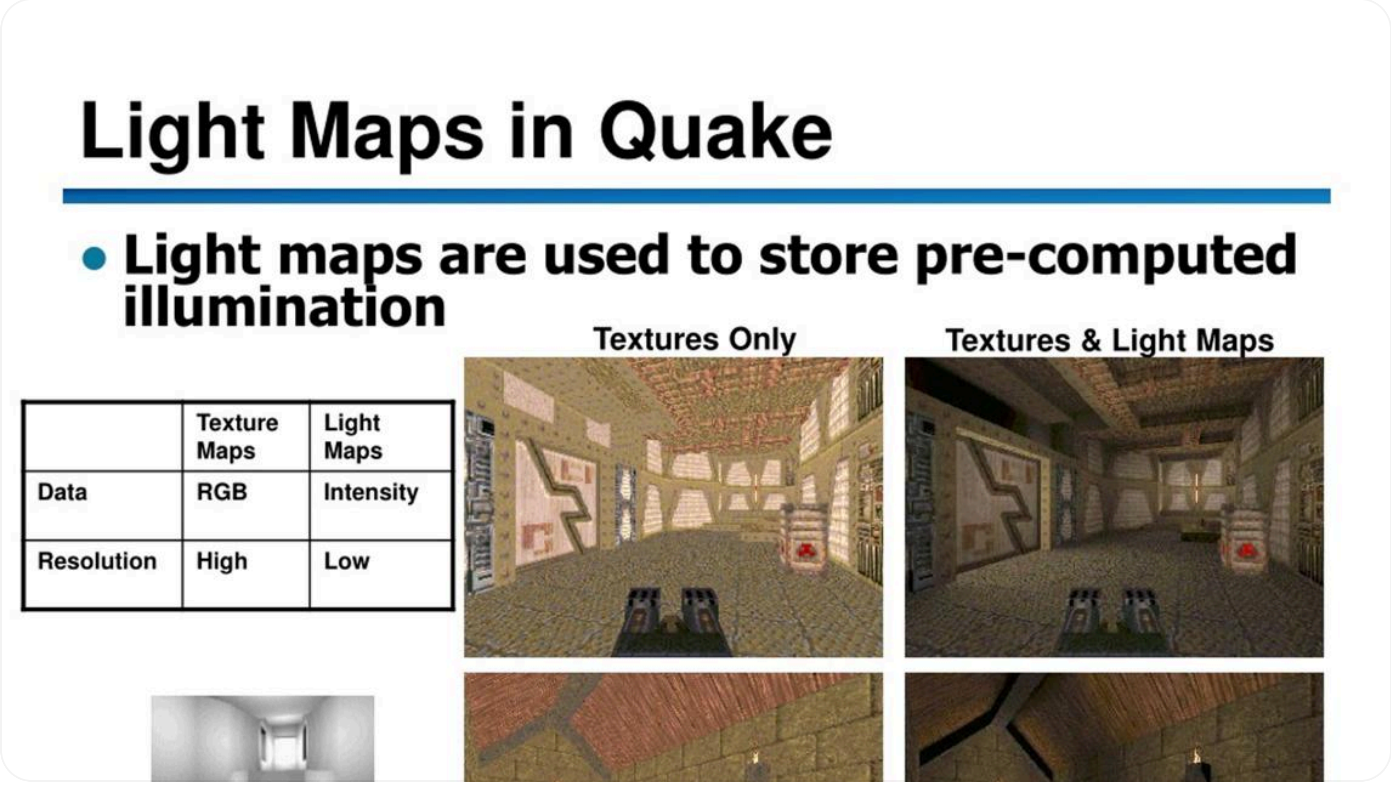
In a conventional Quake BSP:

- Faces share lightmap atlases.
- Lightmap coordinates are derived from the face's texture mapping axes.
- Lightmap density is typically around 16 world units per luxel.
- The lightmap samples are packed into rectangular atlas blocks.

The original face geometry and texture mapping determine where the lightmap samples are placed.

2. What decoupled lightmaps change

With `-world_units_per_luxel`, ericw-tools can generate an independent lightmap projection for each face.



For each face, it must:

1. Calculate a suitable lightmap projection.
2. Determine the face's lightmap extents at the requested density.
3. Allocate a new sample grid.
4. Calculate lighting at the new luxel positions.
5. Store the independent projection axes and dimensions in `DECOUPLED_LM`.
6. Write the resulting lightmap samples into the BSP.

This is a new lighting calculation, not an image resize.

3. Why does it take so long?

The biggest reason is that ericw-tools **recalculates the lighting**.

For each luxel, the lighting compiler may need to evaluate:

- Direct light contributions.
- Shadow visibility and occlusion.
- Indirect light and bounce contributions, depending on compiler settings.
- Multiple light styles.
- RGB light contributions for the `.lit` output.

It has to do this across the BSP, not just upscale one existing texture.

And higher resolution gets expensive quickly:

Resolution	Approximate luxels
16 WU/luxel	1×
8 WU/luxel	4×
4 WU/luxel	16×
2 WU/luxel	64×
1 WU/luxel	256×

These are relative sample counts for the same surface area, not guaranteed runtime multipliers.

4. Could we make this faster?

Potentially, yes. But we need to distinguish two different operations:

A. True decoupled lightmap generation

Calculate new lighting at the requested sample positions. This produces genuinely higher-resolution lighting.

B. Upscaling existing lightmaps

Interpolate existing samples into a denser grid. Much faster, but it does not create new lighting information. It merely makes the existing information denser.

For QSS, I would keep true generation as the primary option.

My architectural suggestion

Eventually, QSS could offer two modes:

- **Recalculate lighting:** Full ericw-tools generation.
- **Resample existing lighting:** Faster conversion for cases where users just want a denser editable lightmap.

The second mode would need careful handling of projection axes and sample interpolation.

One important question before we design this: When you run generation at 16, 8 and 4 WU/luxel, is the entire operation taking a long time, or does it spend most of its time in the actual `light` compiler?

That distinction will tell us where optimization is worthwhile.

You said:

When we come to the point were we start to imitate ericws lighting with
blender, a function to just apply a larger empty atlas to be filled by blender
would be welcome but for now i guess its ok to wait a bit.

I chose a random level (e3m4) and decoupled it for 8, 4, 2 and 1. The latter
was not generated. Seems like 2 is the lowest it can get. The rest runs fine in
ironwail.

ChatGPT said:

That's a very useful experiment! And I agree with your thinking about the future Blender lighting workflow.

There are really two separate capabilities we should eventually support:

1. **Decoupling existing lighting:** Create a larger, independent lightmap sample grid that Blender can fill or edit.
2. **Generating lighting from scratch:** Reproduce ericw-tools' lighting calculations inside Blender.

The first is much simpler and could become a useful intermediate workflow. The second is a considerably larger undertaking involving light transport, visibility, shadowing and potentially radiosity.

For now, using ericw-tools as the lighting compiler is the right decision.

Your e3m4 results

Three resolutions successfully tested in Ironwail

Density	Result
8 WU/luxel	Success
4 WU/luxel	Success
2 WU/luxel	Success
1 WU/luxel	Not generated

The important finding is that 2 WU/luxel works on a real, considerably more complex map.

Why 1 WU/luxel might fail

At 1 WU/luxel, e3m4 needs four times as many samples as at 2 WU/luxel.

Possible limits include:

- Maximum lightmap dimensions or face extents.
- BSP lightdata size limits.
- Light compiler memory consumption.
- Maximum allocation or sample-offset constraints.

I wouldn't assume yet that 2 WU/luxel is a hard limit. We need the compiler's error message to know why 1 failed.

What I'd like to do next

Keep the four test configurations as reference cases, especially 2 WU/luxel.

Then I'd like to inspect the QSS implementation for any assumptions about maximum decoupled face dimensions and sample counts. We should make sure QSS isn't imposing an unnecessary limit of its own.

One question: When 1 WU/luxel failed, did ericw-tools report an error, or did it simply stop without producing the BSP?

That will help us distinguish a compiler limitation from a QSS limitation. 😁

You said:

that should be the log:
---- light / ericw-tools 2.0.0-alpha11 ----

```
LoadBSPFile:
'/mnt/sde1/FPS/IdTech2/QuakeDev/id1/maps/e3m4_decoupled.bsp'
Loaded
'/mnt/sde1/FPS/IdTech2/QuakeDev/id1/maps/e3m4_decoupled.bsp' from archive ''
BSP is version Quake BSP bsp29
Added directory '/mnt/sde1/FPS/IdTech2/QuakeDev/id1/maps'
INFO: using built-in palette.
WARNING: entity at () (worldspawn) has empty key/value "" "0"
WARNING: brush entity with mins [1881 -1407 -191] maxs [1887 -1281 -105] (trigger_once) has empty key/value "targetname" ""
944 entities read, 539 are lights.

--- Options Summary ---
   "verbose" was set to "1" (from command line)
   "gamedir" was set to "/mnt/sde1/FPS/IdTech2/QuakeDev/id1" (from command line)
   "basedir" was set to "/mnt/sde1/FPS/IdTech2/QuakeDev" (from command line)
   "dirt" was set to "1" (from map)
   "dirtmode" was set to "1.000000" (from map)
   "dirtdepth" was set to "192.000000" (from map)
   "dirtscale" was set to "1.000000" (from map)
   "dirtgain" was set to "1.000000" (from map)
   "minlight_dirt" was set to "1" (from command line)
   "bounce" was set to "16" (from map)
   "bouncestyled" was set to "1" (from map)
   "bouncescale" was set to "0.500000" (from map)
   "sunlight" was set to "1.000000" (from map)
   "sunlight_dirt" was set to "1.000000" (from command line)
   "sunlight2_dirt" was set to "1.000000" (from command line)
   "sunlight_penumbra" was set to "4.000000" (from map)
   "world_units_per_luxel" was set to "1.000000" (from command line)

Embree_TraceInit: Embree version: 4.4.0
Embree_TraceInit:
   142 sky faces
   5328 solid faces
   0 filtered faces
   0 shadow-casting skip faces
```

```
SetupLights: 539 initial lights
Creating surface lights for texture "light3_8" from template at
(880 -120 16)
Creating surface lights for texture "light3_7" from template at
(880 -104 16)
Creating surface lights for texture "light3_6" from template at
(880 -88 16)
Creating surface lights for texture "light3_5" from template at
(880 -72 16)
Creating surface lights for texture "light3_3" from template at
(880 -56 16)
Creating surface lights for texture "light1_4" from template at
(880 -40 16)
Creating surface lights for texture "key03_1" from template at
(880 -80 0)
WARNING: no faces found with texture key03_1 (qbsp may have
been run with .wad's missing?)
Creating surface lights for texture "key03_2" from template at
(880 -80 -16)
WARNING: no faces found with texture key03_2 (qbsp may have
been run with .wad's missing?)
Creating surface lights for texture "rune_a" from template at (880
-80 -32)
WARNING: no faces found with texture rune_a (qbsp may have
been run with .wad's missing?)
Creating surface lights for texture "rune2_4" from template at
(880 -80 -48)
Creating surface lights for texture "rune2_5" from template at
(880 -80 -64)
Creating surface lights for texture "metal6_3" from template at
(880 -80 -80)
Creating surface lights for texture "z_exit" from template at (880
-80 -96)
Creating surface lights for texture "*lava1" from template at (880
-80 32)
Creating surface lights for texture "light1_8" from template at
(880 -24 16)
SetupLights: 901 after surface lights
SetupLights: 901 after jittering
WARNING: couldn't nudge light out of solid at 512 -1256 -32
WARNING: couldn't nudge light out of solid at 872 -1224 -32
WARNING: couldn't nudge light out of solid at -176 -2104 328
```

Final count: 901 lights, 64 suns in use.
48 dirtmap vectors
0 faces for smoothing
0 surface light points in use.

ChatGPT said:

Thanks! This log is very helpful. I can already narrow down where the problem occurs.

The important observation: the log does not show an actual error. It simply stops very early in the lighting process.

What the log tells us

The compiler successfully:

- Loaded e3m4 as BSP29.
- Read 944 entities, including 539 lights.
- Initialized Embree 4.4.0.
- Identified 5,328 solid faces.
- Created 901 lights after surface-light processing.
- Accepted `world_units_per_luxel = 1.0`.

So the command-line option itself is working.

What about the warnings?

These are unlikely to explain the failure:

Warning	Meaning
Empty worldspawn key	Map entity metadata issue
Empty trigger targetname	Entity metadata issue
Missing texture faces	Surface-light templates reference absent textures
Couldn't nudge light out of solid	Some lights are embedded in geometry

None of these is an explicit fatal error.

Where things get interesting

The log ends at:

```
0 surface light points in use.
```

We haven't even reached the usual lightmap calculation and output stages.

My suspicion is that the compiler is encountering a resource problem at the very high resolution, possibly memory consumption. At 1 WU/luxel, the sample count is 256 times that of 16 WU/luxel.

But I don't want to diagnose an out-of-memory failure without evidence.

Can you give me two more details?

1. Does the QSS report show a non-zero process exit code, or does it say the process was terminated?
2. Does the complete terminal output contain anything after `0 surface light points in use.`?

If the process was killed by Linux, we can also check:

```
dmesg -T | grep -i -E 'killed process|out of memory'
```

My current assessment: 2 WU/luxel is a proven working resolution for e3m4. We haven't established a hard minimum of 2 yet. Let's find out whether 1 WU/luxel hits a memory limit before changing anything in QSS.

You said:

> hmm.. halfing the wu/luxel rougly doubles the filesize. the 2 wu/luxel has ca. 10mb so the 1wu/luxel should be roughly 20mb. Is this enough for memory concerns?

ChatGPT said:

You're absolutely right to question that. And your file-size observation is useful evidence.

A 20 MB BSP is not, by itself, a convincing reason to suspect memory exhaustion. However, the final BSP size and the compiler's peak memory consumption are two very different things.

Here's why.

1. BSP size versus working memory

The 20 MB estimate describes the stored output.

During lighting calculations, ericw-tools also needs memory for:

- Geometry and acceleration structures (Embree).
- Light sources and their properties.
- Visibility and shadow calculations.
- Intermediate lightmap buffers.
- Bounce-lighting calculations.
- Light styles and RGB sample data.

Some of these structures can consume considerably more memory than the final BSP.

2. Your resolution scaling

Your observations are consistent with the expected sample growth:

Resolution	Relative samples	Approx. BSP size
16 WU/luxel	1×	Reference
8 WU/luxel	4×	—
4 WU/luxel	16×	—
2 WU/luxel	64×	~10 MB
1 WU/luxel	256×	~20 MB expected

Going from 2 to 1 WU/luxel means approximately four times as many lightmap samples, not twice as many. Your estimated file size doubling is therefore worth investigating, although other BSP data and compression/layout effects can influence the result.

3. I want to check something else first

Your log stops at:

```
0 surface light points in use.
```

That is before the main lighting calculation has visibly started.

I'd rather not assume an OOM issue yet.

Please run this immediately after another failed 1 WU/luxel attempt:

```
journalctl -k -b | grep -i -E 'oom|out of memory|killed process'
```

Also check the available memory:

```
free -h
```

If there is no OOM event, we'll investigate a possible compiler crash or another internal limit.

My position: I don't want to impose a 2 WU/luxel restriction on QSS based on one incomplete compiler run. Let's establish the actual cause first.

You said:

> the thing is it seemed to run to the end. it took about double as long as 2wulu took until blender was responding again.

ChatGPT said:

Ah! That changes my interpretation considerably.

If Blender became responsive again after roughly twice the runtime of the 2 WU/luxel run, the process may actually have completed. The log you pasted might simply be incomplete.

And there is another possibility: **QSS's post-generation verification may be rejecting the resulting BSP**, even though ericw-tools successfully generated it.

Let's distinguish those two stages.

Please check three things

1. Look in the e3m4 directory. Is there an `e3m4_decoupled_001.bsp` or another newly numbered output file?
2. Is there a corresponding `.lit` file?
3. Open Blender's Text datablock `QSS_Decoupled_Lightmap_Generation`. What does it say at the bottom?

In particular, look for:

- `VERIFICATION: PASS`
- A compiler exit code.
- Any error after the compiler output.

Why this matters

The generator has two distinct stages:

1. ericw-tools calculates and writes the new BSP.
2. QSS checks the resulting BSP and verifies its `DECOUPLED_LM` data.

If stage 1 succeeds but stage 2 fails, we have a QSS validation issue, not necessarily a lighting compiler limitation.

I'd really like to get 1 WU/luxel working on e3m4. It would be a valuable stress test for QSS. 😁

You said:

ahh Just found this:
ericw-tools light failed (TIMEOUT): t: 00:03:17
[19%] est: 00:03:32
[20%] est: 00:03:37
[21%] est: 00:03:52
[22%] est: 00:04:12
[23%] est: 00:04:29
[24%] est: 00:04:53
[25%] est: 00:04:56
[26%] est: 00:04:54
[27%] est: 00:

ChatGPT said:

Aha! Found it! 😁

This is a QSS timeout, not an ericw-tools resolution limit.

The important part is:

```
ericw-tools light failed (TIMEOUT)
```

The compiler was still working. It had reached approximately 27% and was estimating another few minutes.

What's happening?

QSS currently imposes a maximum execution time on the external `light` process.

At 1 WU/luxel, e3m4 simply takes longer than that limit.

Your observations now make sense:

- 2 WU/luxel completes.
- 1 WU/luxel takes longer.
- Blender becomes responsive again because QSS terminates the process after the timeout.

My proposed fix

I'd improve the generator in three ways:

1. **Configurable timeout:** Let users choose how long the lighting compiler may run.
2. **Better progress reporting:** Show elapsed time, compiler progress and timeout information.
3. **No premature termination:** Allow long-running lighting calculations without imposing an unnecessarily short fixed limit.

I'd also make the default timeout generous enough for large Quake maps.

One thing I'd like to do

I'd prefer to inspect the current timeout implementation in QSS before changing it. We should also check whether the process output is being captured and reported correctly.

My recommendation: remove the arbitrary practical restriction, not the ability to generate high-resolution lightmaps.

Let's make QSS patient enough for serious maps. 😁

You said:

Ok but that is not of high priority. Lets keep it in mind as an to do but for now lets finally go from sidepanel to full workspace.

Worked for 40s

ChatGPT said:

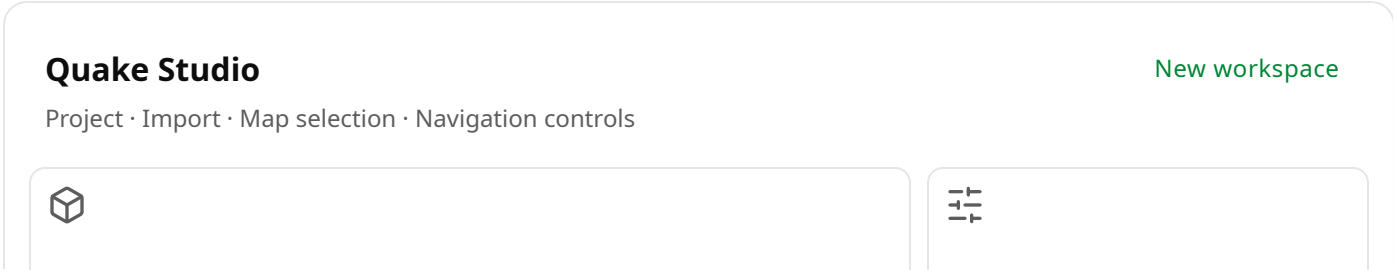
Agreed. We'll park the light-generation timeout as a follow-up and focus on the larger UI transition: moving QSS from its current sidebar-based interface into a dedicated Blender workspace.

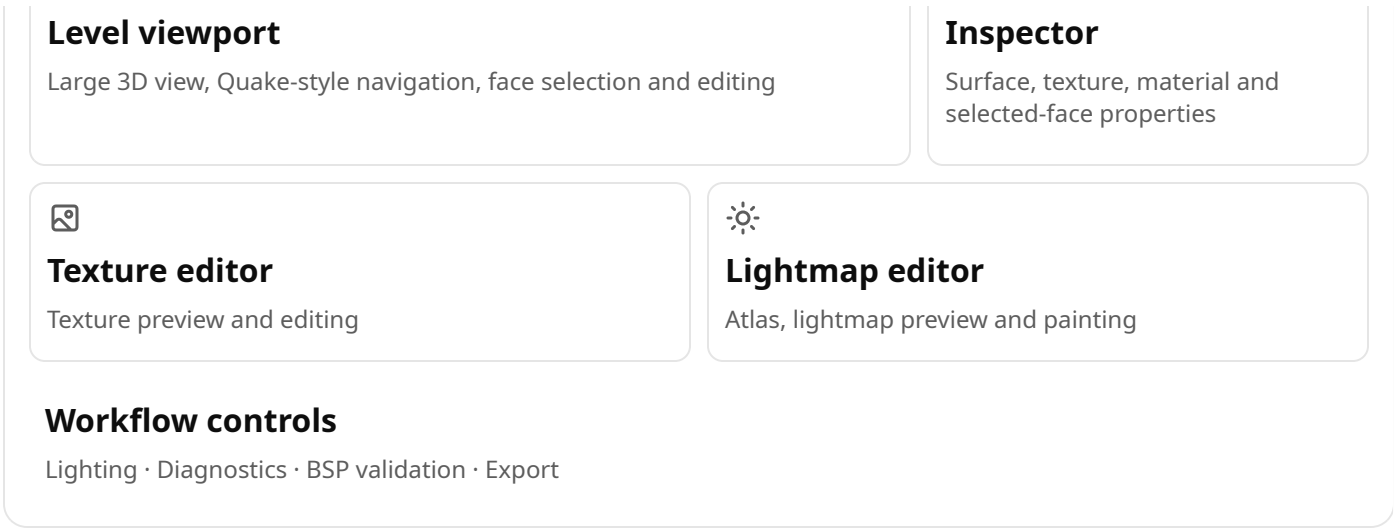
I'll first inspect the current UI layout and registration code on the feature branch so the workspace design builds on what already exists. I'll keep `main` untouched and the PR unmerged.

The current implementation is a good starting point: QSS already has six clearly separated workflows, but they are all `VIEW_3D` sidebar panels. We can reuse much of their existing operators and settings rather than rewrite the tools.

The README already describes the intended direction: a dedicated **Quake Studio** workspace with a large level viewport, image editor, surface/material inspector and lightmap editor.

Proposed Quake Studio layout





One important architectural decision

I recommend adding Quake Studio as a **new workspace**, leaving Blender's Layout, Modeling, Texture Paint and other workspaces intact. The existing sidebar can remain available as a compact fallback during the transition.

I'd build this in stages:

1. **Workspace creation and layout** — create the dedicated workspace and arrange its editors.
2. **Move the existing workflows** — expose import, navigation, texture, lighting, diagnostics and export in the new interface.
3. **Context-aware editors** — make the image and lightmap editors follow the active BSP and selected surface.
4. **Polish and persistence** — ensure the workspace survives saving, restarting Blender and enabling/disabling the add-on.

One decision before implementation: should Quake Studio be an additional workspace alongside Blender's existing workspaces, as I recommend, or should enabling QSS replace the current Layout workspace?